LAPORAN RISET

MODUL 1
"PEMROGRAMAN BASIC ROBOTIKA DENGAN KIT IMCLAB"

oleh :

Evan Adwitiya Dwi Listanto / 23081010093

ROBOTIKA A-801
PROGRAM STUDI INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
JAWA TIMUR
2026

1. Tujuan

Percobaan ini bertujuan untuk :

1. Menghubungkan Perangkat ESP 32 dengan Laptop serta melakukan pengunggahan program dari arduione IDE ke Board ESP
2. Menyalakan serta mematikan motor DC pada board Kit IMC LAB melalui serial dengan perinta “1” Untuk menyalakn dan “0” Untuk Mematikan

2. Dasar Teori

1. **Pin digital ESP32.** Pin GPIO ESP32 dapat diatur sebagai keluaran (OUTPUT) dengan keadaan HIGH atau LOW. Arus yang dapat diberikan pin sangat kecil, sehingga motor DC tidak dihubungkan langsung ke pin, melainkan lewat driver motor.
2. **Driver motor.** Driver motor menerima sinyal logis dari ESP32 dan menyalurkan daya ke motor. Pin D27 dan D26 menentukan arah putaran (kombinasi HIGH dan LOW), sedangkan pin Enable menentukan besar daya yang diteruskan ke motor.
3. **PWM.** PWM (Pulse Width Modulation) mengatur daya rata-rata dengan mengubah lebar pulsa dalam satu periode, disebut duty cycle. Dengan resolusi 8 bit, duty bernilai 0 sampai 255. Nilai 205 berarti sinyal berada pada kondisi HIGH sekitar 80,4% dari satu periode (205/255), Semakin besar duty, semakin besar daya rata-rata yang diterima motor.
4. **Komunikasi serial.** ESP32 dan laptop bertukar data lewat serial (UART) dengan kecepatan yang disepakati, yaitu baud rate 115200. Chip USB-UART pada board mengubah sinyal USB laptop menjadi UART, sehingga laptop mengenalinya sebagai port COM. Pada percobaan ini perintah berupa satu karakter: '1' untuk ON dan '0' untuk OFF.

3. Alat, Bahan, dan Konfigurasi Pin

Perangkat Keras yang digunakan dalam Percobaan ini :

- ESP 32
- Modul motor IMCLab
- Kabel USB Data
- Laptop

Perangkat Lunak yang digunakan dalam Percobaan ini :

- Arduino IDE Versi 2.3.10
- Pyhton dengan Jupyther Notebook
- Visual Studi Code
- Versi core ESP32 di Boards Manager : 2.0.17

Fungsi	Pin ESP32	Mode di kode
LED indikator	GPIO 2	OUTPUT
Motor IN1	GPIO 27	OUTPUT
Motor IN2	GPIO 26	OUTPUT
Enable (kanal PWM 0)	GPIO 12	OUTPUT, dihubungkan ke PWM
Sensor RPM	GPIO 13	INPUT_PULLUP (tidak dipakai di modul ini)

Parameter PWM dan komunikasi pada sketch:

Parameter	Nilai
Frekuensi PWM	30.000 Hz
Kanal PWM	0
Resolusi	8 bit (duty 0 sampai 255)
Duty saat motor ON	205 (sekitar 80,4%)
Duty saat motor OFF	0
Baud rate serial	115200

4. Prosedur Percobaan

1. Hubungan Modul ESP 32 Ke Laptop menggunakan kabel USB data.
2. Pastikan Kabel USB yang digunakan mendukung Transfer data, bukan hanya untuk pengisian daya.
3. Periksa Apakah ESP32 sudah terdeteksi pada komputer.
4. Buka aplikasi Arduino IDE.
5. Salin dan tempel Kode berikut ke Arduino IDE.

Kode `on_off_motor.ino`

```
int ledPin = 2;
char myData = 0;

int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;

// Setting PWM properties
const int freq = 30000;
const int pwmChannel = 0;
const int resolution = 8;
int dutyCycle = 200;

const byte pin_rpm = 13;
int volatile rev = 0;
//int rpm = 0;

void setup()
{
  pinMode(ledPin, OUTPUT);
  Serial.begin(115200);
  // sets the pins
  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(enable1Pin, OUTPUT);
  pinMode(pin_rpm, INPUT_PULLUP);
  //  pinMode(pin_rpm, INPUT);
  //attachInterrupt(digitalPinToInterrupt(pin_rpm), isr, RISING);

  // configure LED PWM functionalitites
  ledcSetup(pwmChannel, freq, resolution);

  // attach the channel to the GPIO to be controlled
  ledcAttachPin(enable1Pin, pwmChannel);

  // testing
  Serial.print("Testing DC Motor...");

}

void MotorOn()
{
  // Motor On - Move DC motor forward
  dutyCycle = 205;
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);
  ledcWrite(pwmChannel, dutyCycle);
}

void MotorOff()
{
  dutyCycle = 0;
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);
  ledcWrite(pwmChannel, dutyCycle);
}

void loop()
{
myData = int(Serial.read());

if (myData == '1'){
  Serial.println("LED is on !!!");
```

```
  digitalWrite(ledPin, HIGH);
  MotorOn();
}
else if (myData == '0'){
  Serial.println("LED is off !!!");
  digitalWrite(ledPin, LOW);
  MotorOff();
}
}
```

6. Pilih board ESP32 melalui menu Tools > Board > esp32 > DOIT ESP32 DEVKIT V1 .
7. Pilih port melalu menu Tools > Port >(Pilih port yang sesuai).
8. Klik Tombol Upload
9. Tunggu hingga proses upload selesai.
10. Buka aplikasi Visual Studio Code**.**
11. Pilih menu File > New File untuk membuat file baru.
12. Simpan file dengan menekan Ctrl + S**.**
13. Pada kolom nama file, tulis nama file dengan ekstensi .inpy,
14. Salin dan tempel kode berikut pada Visual Studio Code.

Kode `Tes_IMCLab_pilih_port_otomatis.ipynb`

```
# ============================================================
# Program Kontrol Motor ESP32 via Serial (Auto-Detect Port)
# Mengirim '1' untuk memutar motor, '0' untuk mematikan motor
# ============================================================

import serial
import serial.tools.list_ports
import time
import sys

# ============================================
# KONFIGURASI
# ============================================
BAUDRATE = 115200  # Harus sama dengan Serial.begin() di ESP32

# Kata kunci pada deskripsi port yang menandakan ESP32 / USB-UART
ESP32_KEYWORDS = [
    "cp210", "cp2102", "cp210x",       # Silicon Labs (umum di ESP32
DevKit)
    "ch340", "ch341", "ch34x",         # WCH (umum di ESP32 clone)
    "ftdi", "ft232",                   # FTDI
    "usb-serial", "usb serial",
    "usb uart", "uart bridge",
    "silicon labs",
    "espressif", "esp32",
]

# (VID, PID) yang umum untuk USB-to-UART ESP32
ESP32_VID_PID = {
    (0x10C4, 0xEA60),  # Silicon Labs CP210x
    (0x10C4, 0xEA70),  # Silicon Labs CP2105
```

```
    (0x1A86, 0x7523),  # WCH CH340
    (0x1A86, 0x55D4),  # WCH CH9102
    (0x0403, 0x6001),  # FTDI FT232
    (0x0403, 0x6015),  # FTDI FT231X
    (0x303A, 0x1001),  # Espressif USB JTAG/Serial
}


# ============================================
# Fungsi untuk melihat daftar port yang tersedia
# ============================================
def list_serial_ports():
    """Menampilkan semua port serial yang terdeteksi."""
    ports = serial.tools.list_ports.comports()
    if not ports:
        print("Tidak ada port serial yang terdeteksi.")
        return []
    print("Port serial yang tersedia:")
    for p in ports:
        vid_pid = ""
        if p.vid is not None and p.pid is not None:
            vid_pid = f" [VID:PID={p.vid:04X}:{p.pid:04X}]"
        print(f"  - {p.device} : {p.description}{vid_pid}")
    return list(ports)


# ============================================
# Fungsi auto-detect port ESP32
# ============================================
def deteksi_port_esp32(verbose=True):
    """
    Mendeteksi port serial ESP32 secara otomatis.
    Return: nama port (str) atau None jika tidak ditemukan.
    """
    ports = serial.tools.list_ports.comports()
    if not ports:
        if verbose:
            print("[!] Tidak ada port serial yang terdeteksi.")
        return None

    kandidat = []

    for p in ports:
        desc = (p.description or "").lower()
        mfg = (p.manufacturer or "").lower()
        hwid = (p.hwid or "").lower()
        teks = f"{desc} {mfg} {hwid}"

        skor = 0

        # 1) Cocokkan VID:PID
        if p.vid is not None and p.pid is not None:
            if (p.vid, p.pid) in ESP32_VID_PID:
                skor += 100

        # 2) Cocokkan kata kunci deskripsi/manufacturer
        for kw in ESP32_KEYWORDS:
            if kw in teks:
                skor += 50
                break

        # 3) Hindari port Bluetooth (sering muncul di Windows)
        if "bluetooth" in teks:
```

```
            skor -= 200

        if skor > 0:
            kandidat.append((skor, p))

    if not kandidat:
        # Fallback: kalau cuma ada 1 port non-bluetooth, pakai itu
        non_bt = [p for p in ports
                  if "bluetooth" not in (p.description or
"").lower()]
        if len(non_bt) == 1:
            if verbose:
                print(f"[i] Tidak ada ciri ESP32 yang cocok, "
                      f"tapi hanya ada 1 port non-Bluetooth:
{non_bt[0].device}")
            return non_bt[0].device

        if verbose:
            print("[!] Port ESP32 tidak dapat dideteksi otomatis.")
            print("    Silakan tentukan PORT secara manual.")
        return None

    # Urutkan berdasarkan skor tertinggi
    kandidat.sort(key=lambda x: x[0], reverse=True)

    if verbose:
        print("Kandidat port ESP32 (skor tertinggi = paling
mungkin):")
        for skor, p in kandidat:
            print(f"  [{skor:>4}] {p.device} : {p.description}")

    return kandidat[0][1].device


# ============================================
# Membuka koneksi serial
# ============================================
def buka_koneksi(port, baudrate=115200):
    """Membuka koneksi serial ke ESP32."""
    try:
        ser = serial.Serial(port, baudrate, timeout=1)
        time.sleep(2)  # Tunggu ESP32 reset setelah koneksi dibuka
        print(f"Terhubung ke ESP32 di {port} @ {baudrate} baud")
        return ser
    except serial.SerialException as e:
        print(f"Gagal membuka port {port}: {e}")
        return None


# ============================================
# Fungsi kirim perintah ke ESP32
# ============================================
def kirim_perintah(ser, perintah: str):
    """
    Mengirim perintah ke ESP32.
    perintah: '1' untuk ON, '0' untuk OFF
    """
    if perintah not in ('0', '1'):
        raise ValueError("Perintah harus '0' atau '1'")

    ser.write(perintah.encode('utf-8'))
    ser.flush()
    time.sleep(0.1)
```

```
    # Baca balasan dari ESP32 (opsional)
    if ser.in_waiting > 0:
        balasan = ser.readline().decode('utf-8',
errors='ignore').strip()
        if balasan:
            print(f"ESP32: {balasan}")


def motor_on(ser):
    """Menyalakan motor (kirim '1')."""
    print(">> Mengirim '1' : Motor ON")
    kirim_perintah(ser, '1')


def motor_off(ser):
    """Mematikan motor (kirim '0')."""
    print(">> Mengirim '0' : Motor OFF")
    kirim_perintah(ser, '0')


# ============================================
# PROGRAM UTAMA
# ============================================
if __name__ == "__main__":

    # 1) Tampilkan port yang tersedia
    print("=" * 50)
    list_serial_ports()
    print("=" * 50)

    # 2) Auto-detect port ESP32
    PORT = deteksi_port_esp32()
    if PORT is None:
        print("Program dihentikan: port ESP32 tidak ditemukan.")
        sys.exit(1)

    print(f"\n>> Port ESP32 terpilih: {PORT}")

    # 3) Buka koneksi serial
    esp32 = buka_koneksi(PORT, BAUDRATE)

    if esp32 is None:
        print("Koneksi gagal. Periksa kembali PORT dan kabel USB.")
    else:
        try:
            # 4) Contoh urutan kontrol motor
            print("\n--- Mulai urutan kontrol motor ---")

            motor_on(esp32)
            time.sleep(3)        # Motor berputar 3 detik

            motor_off(esp32)
            time.sleep(1)        # Jeda 1 detik

            motor_on(esp32)
            time.sleep(2)        # Motor berputar 2 detik

            motor_off(esp32)
            print("--- Urutan kontrol selesai ---\n")

            # 5) Mode interaktif (opsional)
```

```
            print("Mode interaktif: ketik '1'=ON, '0'=OFF,
'q'=keluar")
            while True:
                cmd = input("Perintah: ").strip()
                if cmd == 'q':
                    break
                elif cmd in ('0', '1'):
                    kirim_perintah(esp32, cmd)
                else:
                    print("Perintah tidak valid! Gunakan '1', '0',
atau 'q'.")

        except KeyboardInterrupt:
            print("\nDihentikan oleh pengguna.")

        finally:
            # 6) Pastikan motor mati & tutup koneksi
            try:
                motor_off(esp32)
            except Exception:
                pass
            esp32.close()
            print("Koneksi serial ditutup.")
```

15. Klik Run pada sel pertama untuk menjalankan proses pemilihan port otomatis.
16. Pastikan notebook berhasil mendeteksi port ESP32 yang terhubung.
17. Notebook akan membuka koneksi serial dengan **baud rate 115200**.
18. Notebook menjalankan urutan pengujian motor:
    a. Motor ON selama 3 detik
    b. Motor OFF selama 1 detik
    c. Motor ON selama 2 detik
    d. Motor OFF
19. Pada mode interaktif, ketik 1 untuk ON, 0 untuk OFF, dan q untuk keluar.
20. Catat balasan ESP32 yang tampil di notebook dan amati putaran motor

5. Hasil Percobaan

1. Koneksi dan Balasan ESP32

   Saat Menjalankan File `Tes_IMCLab_pilih_port_otomatis.ipynb` tampil Output sebagai Berikut

```
================================================
Port serial yang tersedia:
  - COM9 : Silicon Labs CP210x USB to UART Bridge (COM9) [VID:PID=10C4:EA60]
  - COM6 : Standard Serial over Bluetooth link (COM6)
  - COM3 : Standard Serial over Bluetooth link (COM3)
  - COM7 : Standard Serial over Bluetooth link (COM7)
  - COM4 : Standard Serial over Bluetooth link (COM4)
  - COM5 : Standard Serial over Bluetooth link (COM5)
  - COM8 : Standard Serial over Bluetooth link (COM8)
================================================
Kandidat port ESP32 (skor tertinggi = paling mungkin):
  [ 150] COM9 : Silicon Labs CP210x USB to UART Bridge (COM9)

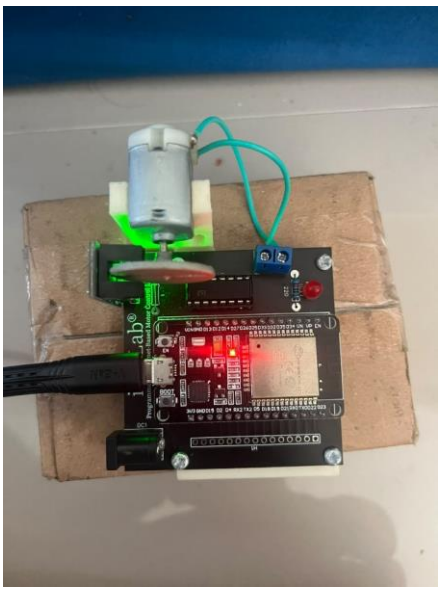
>> Port ESP32 terpilih: COM9
Terhubung ke ESP32 di COM9 @ 115200 baud

--- Mulai urutan kontrol motor ---
>> Mengirim '1' : Motor ON
ESP32: ts Jun  8 201Testing DC Motor...LED is on !!!
>> Mengirim '0' : Motor OFF
ESP32: LED is off !!!
>> Mengirim '1' : Motor ON
ESP32: LED is on !!!
>> Mengirim '0' : Motor OFF
ESP32: LED is off !!!
...
ESP32: LED is off !!!
>> Mengirim '0' : Motor OFF
ESP32: LED is off !!!
Koneksi serial ditutup.
```

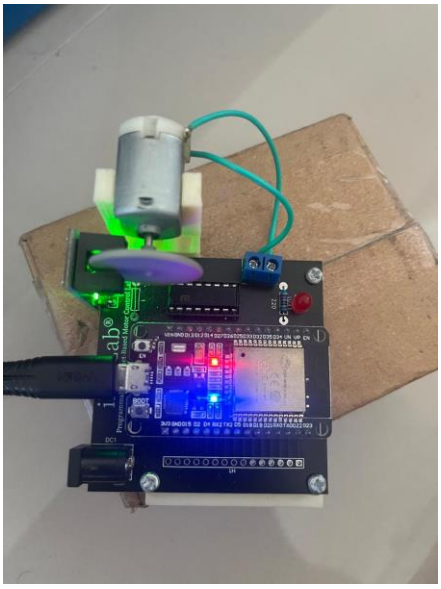
Sistem deteksi otomatis port pada program `Tes_IMCLab_pilih_port_otomatis.ipynb` Berhasil menetapkan port COM9 sebagai port ESP32 dengan skor 150 (terdiri dari 100 poin kecocokan VID:PID dan 50 poin dari kata kunci). Setiap perintah yang dikirimkan ke ESP32 berhasil mendapatkan balasan.

Commented [H1]:

Perintah	Balasan ESP	Durasi DI Kode	Pengamatan Motor
1 (otomatis)	`ts Jun 8 201Testing DC Motor...LED is on !!!`	3 detik	3 Detik
0 (otomatis)	`LED is off !!!`	1 detik	1 detik
1 (otomatis)	`LED is on !!!`	2 detik	2 detik
0 (otomatis)	`LED is off !!!`	-	-
1 (interaktif)	`LED is on !!!`	-	-
0 (interaktif)	`LED is on !!!`	-	-
q (keluar)	`Koneksi serial ditutup.`	-	-



Gamabar 1 Foto Saat Kondisi Perintah “0” lampu dan motor tidak menyala



Gamabar 2 Foto Saat Kondisi Perintah “1” lampu dan motor menyala

6. Analisis

1. Pada kode program Arduino, perintah `Serial.print("Testing DC Motor...")` yang diletakkan di dalam fungsi `setup()` tidak diakhiri dengan karakter baris baru (*newline* atau `\n`). Akibatnya, teks tersebut akan menempel langsung dengan teks balasan atau *output* berikutnya. Pesan ini muncul setiap kali papan ESP32 melakukan proses *reset* otomatis, yang salah satunya dipicu saat koneksi port serial dibuka oleh program Python.
2. Besar daya saat ON. Duty 205 dari 255 berarti sekitar 80,4%, sehingga motor tidak mendapat daya penuh Nilai awal dutyCycle = 200 pada deklarasi variabel tidak pernah dipakai, karena MotorOn() menimpanya menjadi 205 sebelum ledcWrite() pertama dijalankan
3. Satu karakter, satu tindakan. Serial.read() mengembalikan -1 bila tidak ada data. Karena myData ditimpa pada setiap putaran loop(), kondisi '1' atau '0' hanya benar pada putaran saat karakter itu tiba. Karena itu setiap perintah menghasilkan tepat satu balasan.

4. Pesan balasan. Balasan 'LED is on/off' mengacu pada LED, padahal yang dikontrol motor. Ini tidak salah karena LED menjadi indikator status motor, tetapi pesan seperti 'Motor ON' akan lebih jelas bagi pengguna lain.

7. Kesimpulan

1. Program on_off_Motor.ino berhasil diunggah ke ESP32 dan ESP32 berkomunikasi dengan laptop lewat serial 115200 baud, terbukti dari balasan pada setiap perintah.
2. Perintah '1' dan '0' menjalankan dan menghentikan motor dengan duty 205 dan 0.